DUNE – A VM MANAGER FOR MIRAGEOS UNIKERNELS

BY

ANIRUDHA BOSE
ADYASHA PANDA
SUBHASHISH PRADHAN

UNDER THE GUIDANCE OF

PROF. MUKTIKANTA SAHU

in partial fulfillment for the award of the degree of
Bachelor in Technology
in
Computer Science & Engineering

Department of Computer Science & Engineering
International Institute of Information Technology
Bhubaneswar, India

2016, May

Bose A.[ζ], Panda A.[†], Pradhan S.[‡]: *Dune – a VM manager for MirageOS unikernels*, International Institute of Information Technology, Bhubaneswar, © 2016, May.

SOURCE CODE:
http://github.com/onyb/dune

[ζ]ani07nov@gmail.com
[†]adyashapanda@gmail.com
[‡]sprkv5@gmail.com

Source code of this project is licensed under Apache License Version 2.0

CERTIFICATE

Department of Computer Science & Engineering
International Institute of Information Technology
Bhubaneswar, India

This is to certify that the dissertation entitled **Dune – a VM manager for MirageOS unikernels** submitted by Anirudha Bose, Adyasha Panda and Subhashish Pradhan, bearing student IDs B112006, B112002 and B112053 respectively is approved for the award of Degree of Bachelor of Technology in Computer Science & Engineering.

Signature:

Date:

Prof. Muktikanta Sahu
Department of Computer Science & Engineering
International Institute of Information Technology
Bhubaneswar, India

Signature:

Date:

The Coordinator
Department of CSE & IT
International Institute of Information Technology
Bhubaneswar, India

ACKNOWLEDGEMENTS

First of all, we wish to thank our project guide Prof. Muktikanta Sahu for his invaluable guidance in this project, the detailed explanations, the patience and the precision in the suggestions. Second, we wish to thank the Department of Computer Science for facilitating the work undertaken in the major thesis. Our sincere thanks to all the people who have contributed to this thesis through their research and knowledge.

ABSTRACT

There has been a recent upsurge of interest in container technologies, especially unikernels like ClickOS, OSv [Kivity et al., 2014], IncludeOS [Bratterud et al., 2015], runtime.js, etc. Unikernels are specialised, single-address-space machine images constructed by using library operating systems.

These specialized operating systems can be constructed in many different ways for many different applications on heterogeneous hardware platforms. Some systems (like Rumprun) are language-agnostic, and provide a platform for any application code based on the requests it makes to the operating system. Others (like MirageOS and HaLVM) leverage high-level languages and a runtime to provide an API for operating system functionality. Unikernels can run in containers, on hypervisors, and on a wide array of bare-metal hardware.

We present a VM manager for application containerization using MirageOS unikernels and provide an API for managing cloud applications deployed on a hypervisor. This API provides an interface to asynchronously register, configure, compile, optimize, start, stop, and destroy unikernels. This technology provides a secure and scalable solution to microservice enabled applications.

Keywords: unikernel, mirage, cloud computing, library OS, virtual machine manager

CONTENTS

1 INTRODUCTION 1

2 MOTIVATION AND OBJECTIVE 3

3 ANYKERNEL AND RUMP KERNEL 5
- 3.1 Monolithic Kernel 5
- 3.2 Challenges with the Monolithic Kernel 5
- 3.3 Anykernel and Rump kernel 6
- 3.4 Details 8
- 3.5 Summary 9

4 UNIKERNEL AND MIRAGEOS 11
- 4.1 Cloud Computing Challenges 11
- 4.2 The Shift to Single-Purpose Appliances 12
- 4.3 Limitations of Current Operating Systems 12
- 4.4 Library Operating System 13
- 4.5 Stronger Programming Abstractions 15
- 4.6 A Functional Prototype in OCaml 16
- 4.7 Why OCaml? 16
- 4.8 Modular Operating-System Libraries 17
- 4.9 Configuration and State 18
- 4.10 Linking the Xen Unikernel 19
- 4.11 Benefits 19
- 4.12 Deployment and Management 20
- 4.13 Resource Efficiency and Customization 21
- 4.14 A New Frontier of Portability 21

5 ARCHITECTURE AND IMPLEMENTATION 23
- 5.1 Architecture Overview 23
- 5.2 Dune API 23
- 5.3 Flask API Endpoints 24
- 5.4 Backends 25
- 5.5 Workers 25
- 5.6 Test Unikernel 26

6 RESULTS AND ANALYSIS 27
- 6.1 Performance analysis 27
- 6.2 Case Study 27

7 CONCLUSION AND FUTURE WORK 29
- 7.1 Conclusion 29
- 7.2 Future work 29

BIBLIOGRAPHY 31

INDEX 31

LIST OF TABLES

Table 1 Approximate sizes of libraries used by a typical MirageOS Xen unikernel running a web server 19
Table 2 Execution time (in ms) of Unikernel backend methods in MirageOS 27
Table 3 Monthly compute hours 28
Table 4 Monthly expenses per usage 28

LIST OF FIGURES

Figure 1 Illustration of client types 7
Figure 2 MirageOS Unikernel Architecture 13
Figure 3 Logical workflow in MirageOS 14
Figure 4 Partial module graph for a static web server 18
Figure 5 Virtual address space of the MirageOS Xen Unikernel Target 20
Figure 6 Architectural overview of Dune 23
Figure 7 Execution time (in ms) of Unikernel backend methods in MirageOS 27
Figure 8 Monthly compute usage hours 28

1 INTRODUCTION

A cloud application is set up on a base that includes an operating system that provides the kernel, drivers, user space libraries and utilities [Kantee, 2012]. This base faces many challenges. First it lacks security due to the interaction with untrusted devices and services and due to the large attack surface that includes the components of the operating system itself. Second the application development, testing and deployment is dependent on the base, the procedure for which is convoluted and tedious.

The industry moved to virtualized solutions like virtual machines and containers that secure the application by isolation in a new operating system base and scale it according to the requirement. However the problem of the large attack surface area and unsafe interaction doesn't go away.

Thus we use a new approach that uses the concept of a minimized base that contains the bare essentials like drivers and supporting libraries, just enough to run an isolated instance of an application. These single application operating systems concepts are implemented as Rump kernels and Unikernels which will form the basis of our framework for API managed Cloud applications.

2 MOTIVATION AND OBJECTIVE

Most applications that run in the cloud aren't optimised to do so. They inherently carry assumptions about the underlying operating system with them, including vulnerabilities and bloat. Compartmentalisation of large servers into smaller VMs has enabled many new businesses to get started and achieve scale. This has been great for new services but many of those virtual machines are single-purpose and yet they contain largely complete operating systems which themselves run applications like web-servers. This means a large part of the footprint is unused and unnecessary, which is both costly and a security risk (due to the larger attack surface).

Unikernels represent a new approach where only the necessary components of the OS are included and compiled along with the application. This results in highly efficient and extremely lean appliances, with a much smaller attack surface [Madhavapeddy and Scott, 2013]. These appliances can be deployed directly to the cloud and embedded devices, with the benefits of reduced costs and increased security and scalability.

Although unikernels can be executed as ordinary UNIX binaries, it is recommended to run them in a hypervisor, such as Xen. A hypervisor is a program that allows multiple operating systems to share a single hardware host. Each operating system appears to have the host's processor, memory, and other resources all to itself. However, the hypervisor is actually controlling the host processor and resources, allocating what is needed to each operating system in turn and making sure that the guest operating systems cannot disrupt each other.

Applications can be packaged using Unikernels and Rump kernels which can thus be deployed on bare metal servers, or anything that supports Xen, including popular cloud services like Amazon EC2, IBM Softlayer, Rackspace Cloud, Linode, etc. These specialized applications are API managed, offer low memory footprint with high scalability and easy deployment.

The objective of this thesis is to evaluate application containerization using Unikernels and to propose a method for providing API managed cloud applications.

3 ANYKERNEL AND RUMP KERNEL

3.1 MONOLITHIC KERNEL

Major contemporary operating systems follow the monolithic kernel model. Of popular general purpose operating systems, for example Linux, Windows and Mac OS X are regarded as monolithic kernel operating systems. A monolithic kernel means that the entire kernel is executed in a single privileged domain, as opposed to being spread out to multiple independent domains which communicate via message passing. The single privileged domain in turn means that all code in the kernel has full capability to directly control anything running on that particular system[Kantee, 2012]. Furthermore, the monolithic kernel does not inherently impose any technical restrictions for the structure of the kernel: a routine may call any other routine in the kernel and access all memory directly. However, like in most disciplines, a well-designed architecture is desirable. Therefore, even a monolithic kernel tends towards structure.

3.2 CHALLENGES WITH THE MONOLITHIC KERNEL

Despite its widespread popularity, a number of suboptimal characteristics in monolithic kernels have been identified:

1. **Weak security and robustness** — Since all kernel code runs in the same privileged domain, a single mistake can bring the whole system down. The fragile nature of the monolithic kernel is a long-standing problem to which all monolithic kernel operating systems are vulnerable.

   Bugs are an obvious manifestation of the problem, but there are more subtle issues to consider. For instance, widely used file system drivers are vulnerable against untrusted disk images. This vulnerability is acknowledged in the manual page of the mount command for example on Linux: "It is possible for a corrupted file system to cause a crash". The commonplace act of accessing untrusted removable media such as a USB stick or DVD disk with an in-kernel file system driver opens the entire system to a security vulnerability.

2. **Limited possibilities for code reuse** — The code in a monolithic kernel is viewed to be an all-or-nothing deal due to a belief that everything is intertwined with everything else. This belief implies that features cannot be cherry-picked and put into use in other contexts and that the kernel drivers have value only when they are a part of the monolithic kernel. Examples include file systems and networking.

   One manifestation of this belief is the reimplementation of kernel drivers for userspace. These re-implementations include TCP/IP stacks and file system drivers for the purposes of research, testing, teaching

and application-level drivers. The common approaches for reimplementation are starting from scratch or taking a kernel driver and adjusting the code until the specific driver can run in userspace.

If cherry-picking unmodified drivers were possible, kernel drivers could be directly used at application level. Code reuse would not only save the initial implementation effort, but more importantly it would save from having to maintain the second implementation.

3. **Development and testing is convoluted** — This is a corollary of the previous point: in the general case testing involves booting up the whole operating system for each iteration. Not only is the bootup slow in itself, but when it is combined with the fact that an error may bring the entire system down, development cycles become long and batch mode regression testing is difficult. The difficulty of testing affects how much testing and quality assurance the final software product receives. Users are indirectly impacted: better testing produces a better system.

   Due to the complexity and slowness of in-kernel development, a common approach is to implement a prototype in userspace before porting the code to the kernel. For example FFS in BSD and ZFS in Solaris were implemented this way. This approach may bring additional work when the code is being moved into the kernel, as the support shim in userspace may not have fully emulated all kernel interfaces.

3.3 ANYKERNEL AND RUMP KERNEL

A flexible kernel architecture is needed to solve the challenges faced by monolithic kernels and yet retain the properties of the monolithic kernel. Furthermore, it is possible to implement the flexible kernel architecture solely by good programming principles and without introducing levels of indirection which hinder the monolithic kernel's performance characteristics. An example of such flexible kernel implementation is the BSD-derived open source monolithic kernel OS, NetBSD's implementation of Anykernel and Rump kernel.

An anykernel is defined as an organization of kernel code which allows the kernel's unmodified drivers to be run in various configurations such as application libraries and microkernel style servers, and also as part of a monolithic kernel [Kantee, 2012]. This approach leaves the configuration the driver is used in to be decided at runtime. For example, if maximal performance is required, the driver can be included in a monolithic kernel, but where there is reason to suspect stability or security, the driver can still be used as an isolated, non-privileged server where problems cannot compromised the entire system.

An anykernel can be instantiated into units which virtualize the bare minimum support functionality for kernel drivers. These virtualized kernel instances are called rump kernels since they retain only a part of the original kernel. This minimalistic approach makes rump kernels fast to bootstrap (10ms) and introduces only a small memory overhead (1MB per instance).

At runtime, a rump kernel can assume the role of an application library or that of a server. Programs requesting services from rump kernels are called rump kernel clients. There are three rump kernel client (or simply client) types.

1. **Local** — the rump kernel is used in a library capacity. Like with any library, using the rump kernel as a library requires that the application is written to use APIs provided by a rump kernel. The main API for a local client is a system call API with the same call signatures as on a regular NetBSD system. For example, it is possible to use a kernel file system driver as a library in an application which interprets a file system image.

2. **Microkernel** — the host routes client requests from regular processes to drivers running in isolated servers. Unmodified application binaries can be used. For example, it is possible to run a block device driver as a microkernel style server, with the kernel driver outside the privileged domain.

3. **Remote** — the client and rump kernel are running in different containers (processes) with the client deciding which services to request from the rump kernel and which to request from the host kernel. For example, the client can use the TCP/IP networking services provided by a rump kernel. The kernel and client can exist either on the same host or on different hosts. In this model, both specifically written applications and unmodified applications can use services provided by a rump kernel. The API for specifically written applications is the same as for local clients. For example, it is possible to use an unmodified Firefox web browser with the TCP/IP code running in a rump kernel server.

Each configuration contributes to solving the motivating problems:

1. **Security and robustness** — When necessary, the use of a rump kernel will allow unmodified kernel drivers to be run as isolated microkernel servers while preserving the user experience. At other times the same driver code can be run in the original fashion as part of the monolithic kernel.

2. **Code reuse** — A rump kernel may be used by an application in the same fashion as any other userlevel library. A local client can call any routine inside the rump kernel.

3. **Development and testing** — The lightweight nature and safety properties of a rump kernel allow for safe testing of kernel code with iteration times in the millisecond range. The remote client model enables the creation of tests using familiar tools.

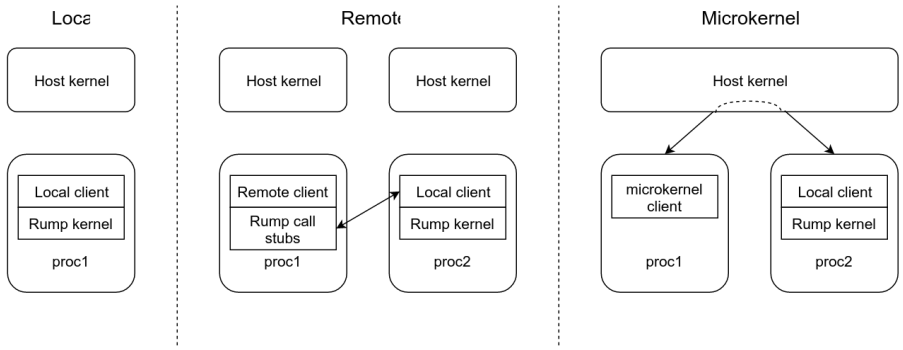

Figure 1: Illustration of client types

3.4 DETAILS

The CPU specific code is on the bottom layer of the OS. This code takes care of low level bootstrap and provides an abstract interface to the hardware. In most, if not all, modern general purpose operating systems the CPU architecture is abstracted away from the bulk of the kernel and only the lowest layers have knowledge of it. The interfaces provided by the CPU specific code are the hypercall interfaces that the OS runs on. In the NetBSD kernel these functions are usually prefixed with "cpu".

The virtual memory subsystem manages the virtual address space of the kernel and processes. Virtual memory management includes defining what happens when a memory address is accessed. Examples include normal read/write access to the memory, flagging a segmentation violation, or a file being read from the file system.

The process execution subsystem understands the formats that executable binaries use and knows how to create a new process when an executable is run.

The scheduling code includes a method and policy to define what code a CPU is executing. The currently executing thread can be switched either when the scheduler decides it has run too long, or when the thread itself makes a system call which requires waiting for a condition to become true before execution can be resumed. When a thread is switched, the scheduler calls the CPU specific code to save the machine context of the current thread and load the context of the new thread. In NetBSD, both user processes and the kernel are preemptively scheduled, meaning the scheduler can decide to unschedule the currently executing thread and schedule a new one.

Atomic operations enable modifying memory atomically and avoid race conditions in for example a read-modify-write cycle. For uniprocessor architectures, kernel atomic operations are a matter of disabling interrupts and preemption for the duration of the operation. Multiprocessor architectures provide machine instructions for atomic operations. The operating system's role with atomic operations is mapping function interfaces to the way atomic operations are implemented on that particular machine architecture.

Synchronization routines such as mutexes and condition variables build upon atomic operations and interface with the scheduler. For example, if locking a mutex is attempted, the condition for it being free is atomically tested and set. If a sleep mutex was already locked, the currently executing thread interfaces with the scheduling code to arrange for itself to be put to sleep until the mutex is released.

Various support interfaces such CPU cross-call, time-related routines, kernel linkers, etc. provide a basis on which to build drivers.

Resource management includes general purpose memory allocation, a pool and slab allocator, file descriptors, PID namespace, etc. Notably, in addition to generic resources such as memory, there are more specific resources to manage.

Drivers interact with external objects such as file system images, hardware, the network, etc. After a fashion, it can be said they accomplish all the useful work an operating system does. It needs to be remembered, though, that they operate by building on top of the entities mentioned earlier in this section. Ultimately the drivers are to be utilized, but to make them available their dependencies must be dealt with. Being able to reuse drivers means semantically equivalent implementations of the support routines have to be

provided that the drivers use. The straightforward way is to run the entire kernel, but it not always the optimal approach.

3.5 SUMMARY

A rump kernel is a partial virtualization of an operating system kernel with the virtualization target being the drivers. To be as lightweight as possible, a rump kernel relies on two features: relegating support functionality to the host where possible and an anykernel codebase where different units of the kernel (e.g. networking and file systems) are disjoint enough to be usable in configurations where all parties are not present.

Rump kernels support three types of clients: local, microkernel and remote. Each client type has its unique properties and varies for example in access rights to a rump kernel, the mechanism for making requests, and performance characteristics. Remote clients are able to access a rump kernel over the Internet.

For drivers to function, a rump kernel must possess runtime context information. This information consists of the process/thread context and a unique rump kernel CPU that each thread is associated with. A rump kernel does not assume virtual memory, and does not provide support for page faults or memory protection. Virtual memory protection and page faults, where necessary, are always left to be performed by the host of the rump kernel client.

4 UNIKERNEL AND MIRAGEOS

4.1 CLOUD COMPUTING CHALLENGES

Cloud computing has been pioneering the business of renting computing resources in large data centers to multiple (and possibly competing) tenants. The basic enabling technology for the cloud is operating-system virtualization such as Xen or VMWare, which allows customers to multiplex VMs (virtual machines) on a shared cluster of physical machines [Madhavapeddy and Scott, 2013]. Each VM presents as a self-contained computer, booting a standard operating-system kernel and running unmodified applications just as if it were executing on a physical machine.

A key driver of the growth of cloud computing in the early days was server consolidation. Existing applications were often installed on physical hosts that were individually underutilized, and virtualization made it feasible to pack them onto fewer hosts without requiring any modifications or code recompilation. VMs are also managed via software APIs rather than physical actions.

They can be centrally backed up and migrated across different physical hosts without interrupting service. Today commercial providers such as Amazon and Rackspace maintain vast data centers that host millions of VMs. These cloud providers relieve their customers of the burden of managing data centers and achieve economies of scale, thereby lowering costs.

While operating-system virtualization is undeniably useful, it adds yet another layer to an already highly layered software stack now including: support for old physical protocols (e.g., disk standards developed in the 1980s, such as IDE); irrelevant optimizations (e.g., disk elevator algorithms on SSD drives); backward-compatible interfaces (e.g., POSIX); user-space processes and threads (in addition to VMs on a hypervisor); and managed-code runtimes (e.g., OCaml, .NET, or Java). All of these layers sit beneath the application code. The situation of operating system virtualization and applications is doomed to adding new layers of indirection and abstraction every few years, leaving future generations of programmers to become virtual archaeologists as they dig through hundreds of layers of software emulation to debug even the simplest applications.

The solution to these virtualization problems can be achieved by restructuring the entire VMs, including all kernel and user-space code into more modular components that are flexible, secure, and reusable in the style of a library operating system. What would the benefits be if all the software layers in an appliance could be compiled within the same high-level language framework instead of dynamically assembling them on every boot? First, some background information about appliances, library operating systems, and type-safe programming languages.

4.2 THE SHIFT TO SINGLE-PURPOSE APPLIANCES

A typical VM running on the cloud today contains a full operating system image: a kernel such as Linux or Windows hosting a primary application running in user space (e.g., MySQL or Apache), along with secondary services (e.g., syslog or NTP) running concurrently. The generic software within each VM is initialized every time the VM is booted by reading configuration files from storage.

Despite containing many flexible layers of software, most deployed VMs ultimately perform a single function such as acting as a database or Web server. The shift toward single-purpose VMs is a reflection of just how easy it has become to deploy a new virtual computer on demand. Even a decade ago, it would have taken more time and money to deploy a single (physical) machine instance, so the single machine would need to run multiple end-user applications and therefore be carefully configured to isolate the constituent services and users from each other.

The software layers that form a VM haven't yet caught up to this trend, and this represents a real opportunity for optimization—not only in terms of performance by adapting the appliance to its task, but also for improving security by eliminating redundant functionality and reducing the attack surface of services running on the public cloud. Doing so statically is a challenge, however, because of the structure of existing operating systems.

4.3 LIMITATIONS OF CURRENT OPERATING SYSTEMS

The modern hypervisor provides a resource abstraction that can be scaled dynamically— both vertically by adding memory and cores, and horizontally by spawning more VMs. Many applications and operating systems can't fully utilize this capability since they were designed before modern hypervisors came about (and the physical analogs such as memory hotplug were never ubiquitous in commodity hardware). Often, external application-level load balancers are added to traditional applications running in VMs in order to make the service respond elastically by spawning new VMs when load increases. Traditional systems, however, are not optimized for size or boot time (Windows might apply a number of patches at boot time, for example), so the load balancer must compensate by keeping idle VMs around to deal with load spikes, wasting resources and money.

These problems with operating systems simply couldn't be fixed because modern operating systems are intended to remain resolutely general-purpose to solve problems for a wide audience. For example, Linux runs on an incredibly diverse set of platforms, from low-power mobile devices to high-end servers powering vast data centers. Compromising this flexibility simply to help one class of users improve application performance would not be acceptable. On the other hand, a specialized server appliance no longer requires an OS to act as a resource multiplexer since the hypervisor can do this at a lower level. One obvious problem with this approach is that most existing code presumes the existence of large but rather calcified interfaces such as POSIX or the Win32 API.

Unikernels are single-purpose appliances[Anil Madhavapeddy, 2015] that are compile-time specialised into standalone kernels, and sealed against

modification when deployed to a cloud platform. In return they offer significant reduction in image sizes, improved efficiency and security, and should reduce operational costs. Unikernels are specialized OS kernels that are written in a high-level language and act as individual software components. A full application (or appliance) consists of a set of running unikernels working together as a distributed system.

The MirageOS unikernel architecture is outlined in figure 2. MirageOS is based on the OCaml language and emits unikernels that run on the Xen hypervisor. To explain how it works, let's look at a radical operating-system architecture from the 1990s that was clearly ahead of its time — libOS.

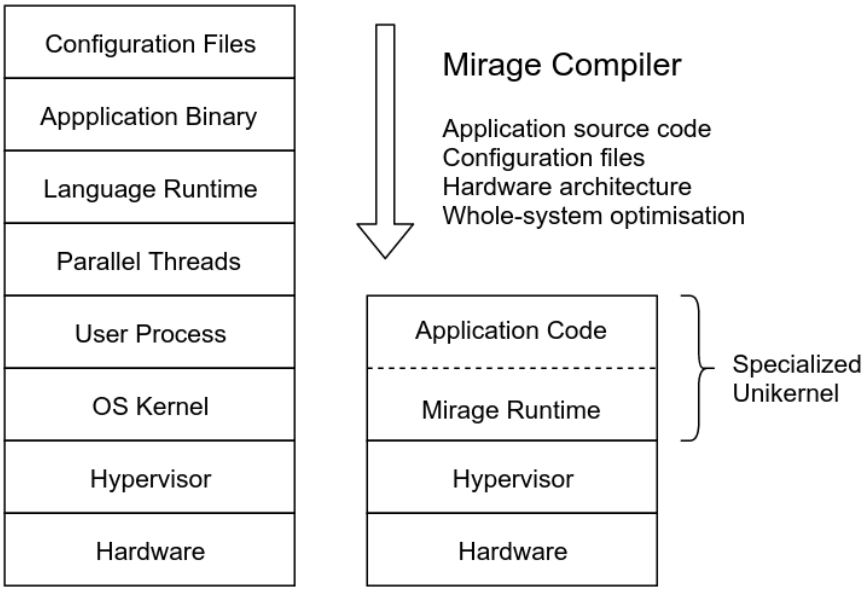

Figure 2: MirageOS Unikernel Architecture

4.4 LIBRARY OPERATING SYSTEM

This is not the first time people have asked these existential questions about operating systems. Several research groups have proposed operating-system designs based on an architecture known as a library operating system (or libOS). The first such systems were Exokernel and Nemesis in the late 1990s. In a libOS, protection boundaries are pushed to the lowest hardware layers, resulting in: (1) a set of libraries that implement mechanisms, such as those needed to drive hardware or talk network protocols; and (2) a set of policies that enforce access control and isolation in the application layer.

The libOS architecture has several advantages over more conventional designs. For applications where performance, and especially predictable performance is required, a libOS wins by allowing applications to access hardware resources directly without having to make repeated privilege transitions to move data between user space and kernel space. The libOS does not have a central networking service into which both high-priority network packets (such as those from a video conference call) and low-priority packets (such as those from a background file download) are forced to mix and interfere. Instead, libOS applications have entirely separate queues, and packets mix together only when they arrive at the network device.

The libOS architecture has two big drawbacks. First, running multiple applications side by side with strong resource isolation is tricky (although Nemesis did an admirable job of minimizing crosstalk between interactive applications). Second, device drivers must be rewritten to fit the new model. Happily, OS virtualization overcomes these drawbacks on commodity hardware. A modern hypervisor provides VMs with CPU time and strongly isolated virtual devices for networking, block storage, USB, and PCI bridges. A libOS running as a VM needs to implement only drivers for these virtual hardware devices and can depend on the hypervisor to drive the real physical hardware. Isolation between libOS applications can be achieved at low cost simply by using the hypervisor to spawn a fresh VM for each distinct application, leaving each VM free to be extremely specialized to its particular purpose. The hypervisor layer imposes a much simpler, less fine-grained policy than a conventional operating system, since it just provides a low-level interface consisting of virtual CPUs and memory pages, rather than the process and file oriented architecture found in conventional operating systems.

Although OS virtualization has made the libOS possible without needing an army of device driver writers, protocol libraries are still needed to replace the services of a traditional operating system. Modern kernels are all written in C, which excels at low-level programs such as device drivers but lacks the abstraction facilities of higher-level languages and demands careful manual tracking of resources such as memory buffers. As a result, many applications contain memory handling bugs, which often manifest as serious security vulnerabilities.

Figure 3 shows the logical workflow in MirageOS. Precise dependency tracking from source code (both local and global libraries) and configuration files lets the full provenance of the deployed kernel binaries be recorded in immutable data stores, sufficient to precisely recompile it on demand.

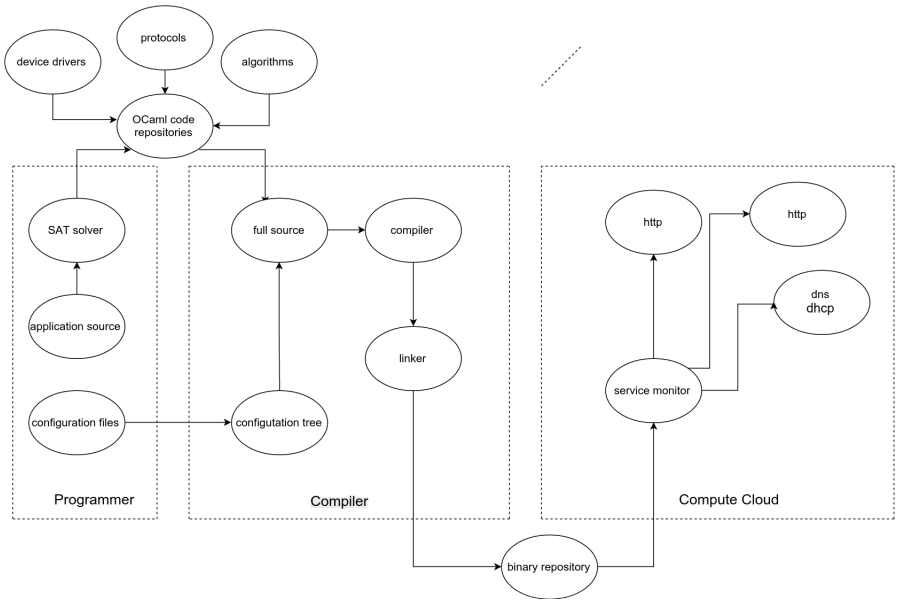

Figure 3: Logical workflow in MirageOS

4.5 STRONGER PROGRAMMING ABSTRACTIONS

High-level languages are steadily gaining ground in general application development and are increasingly used to glue components together via orchestration frameworks (e.g., Puppet and Chef). Unfortunately, all this logic is typically scattered across software components and is written in several languages. As a result, it is hard to reason statically about the whole system's behavior just by analyzing the source code.

MirageOS aims to unify these diverse interfaces, both in kernel and application user spaces, into a single high-level language framework. Some of the benefits of modern programming languages include:

- **Static type checking** — Compilers can classify program variables and functions into types and reject code where a variable of one type is operated on as if it were a different type. Static type checking catches these errors at compile time rather than runtime and provides a flexible way for a systems programmer to protect different parts of a program from each other without depending solely on hardware mechanisms such as virtual memory paging. The most obvious benefit of type checking is the resulting lack of memory errors such as buffer or integer overflows, which are still prevalent in the CERT (Computer Emergency Readiness Team) vulnerability database. A more advanced use is capability-style access control, which can be entirely enforced in a static type system such as ML's, as long as the code all runs within the same language runtime.

- **Automatic memory management** — Runtime systems relieve programmers of the burden of allocating and freeing memory, while still permitting manual management of buffers (e.g., for efficient I/O). Modern garbage collectors are also designed to minimize application interruptions via incremental and generational collection, thus permitting their use in high-performance systems construction.

- **Modules** — When the code base grows, modules partition it into logical components with well-defined interfaces gluing them together. Modules help software development scale as internal implementation details can be abstracted and the scope of a single source-code change can be restricted. Some module systems, such as those found in OCaml and Standard ML, are statically resolved at compilation time and are largely free of runtime costs. The goal is to harness these module systems to build whole systems, crossing traditional kernel and user-space boundaries in one program.

- **Metaprogramming** — If the runtime configuration of a system is partially understood at compile time, then a compiler can optimize the program much more than it would normally be able to. Without knowledge of the runtime configuration, the compiler's hands are tied, as the output program must remain completely generic, just in case. The goal here is to unify configuration and code at compilation time and eliminate waste before deploying to the public cloud.

Together, these features significantly simplify the construction of large-scale systems: managed memory eliminates many resource leaks, type inference results in more succinct source code, static type checking verifies that code matches some abstraction criteria at compilation time rather than

execution time, and module systems allow the manipulation of this code at the scales demanded by a full OS and application stack.

4.6 A FUNCTIONAL PROTOTYPE IN OCAML

The major design decision for building MirageOS was based on unification of the programming models underlying library operating systems and cloudservice deployment. The first design decision was to adopt the principles behind functional programming to construct the prototype. Functional programming has an emphasis on supporting abstractions that make it easier to track mutability in programs, and previous research has shown that this need not come at the price of performance.

The challenge was to identify the correct modular abstractions to support the expression of an entire operating system and application software stack in a single manageable structure. MirageOS has since grown into a mature set of almost 100 open-source libraries that implement a wide array of functionality, and it is starting to be integrated into commercial products such as Citrix XenServer.

Figure 3 illustrates MirageOS's design. It grants the compiler a much broader view of source-code dependencies than a conventional cloud deployment cycle:

- All source-code dependencies of the input application are explicitly tracked, including all the libraries required to implement kernel functionality. MirageOS includes a build system that internally uses a SAT solver (using the OPAM package manager, with solvers from the Mancoosi project) to search for compatible module implementations from a published online package set. Any mismatches in interfaces are caught at compile time because of OCaml's static type checking.

- The compiler can then output a full stand-alone kernel instead of just a Unix executable. These unikernels are single-purpose libOS VMs that perform only the task defined in their application source and configuration files, and they depend on the hypervisor to provide resource multiplexing and isolation. Even the bootloader, which has to set up the virtual memory page tables and initialize the language runtime, is written as a simple library. Each application links to the specific set of libraries it needs and can glue them together in application-specific ways.

- The specialized unikernels are deployed on the public cloud. They have a significantly smaller attack surface than the conventional virtualized equivalents and are more resource-efficient in terms of boot time, binary size, and runtime performance.

4.7 WHY OCAML?

OCaml is the sole base language for MirageOS for a few key reasons. It is a full-fledged systems programming language with a flexible programming model that supports functional, imperative, and object-oriented styles within a single, ML-inspired type system. It also features a portable single-threaded runtime that makes it ideal for porting to restricted environments

such as a barebones Xen VM. The compiler heavily emphasizes static type checking, and the resulting binaries are fast native code with minimal runtime type information. Principal type inference allows type annotations to be safely omitted, and the module system is among the most powerful in a general-purpose programming language in terms of permitting flexible and safe code reuse and refactoring. Finally, there were several examples of large-scale uses of OCaml in industry and within Xen itself, and the positive results were encouraging before embarking on the large multiyear project that MirageOS turned out to be.

4.8 MODULAR OPERATING-SYSTEM LIBRARIES

OCaml supports the definition of module signatures (a collection of datatype and function declarations) that abstract the implementation of module structures (definitions of concrete data types and functions). Modules can be parameterized over signatures, creating functors that define operations across other data types. (For more information about OCaml modules, functors, and objects, see Real World OCaml, published by O'Reilly and available at https://realworldocaml.org.) Thhe OCaml module systemwas applied to breaking the usually monolithic OS kernel functionality into discrete units. This lets programmers build code that can be progressively specialized as it is being written, starting from a process in a familiar Unix environment and ending up with a specialized cloud unikernel running on Xen.

Consider a simple example. Figure 5 shows a partial module graph for a static Web server. Libraries are module graphs that abstract over operating-system functionality, and the OPAM package manager solves constraints over the target architecture. The application MyHomePage depends on an HTTP signature that is provided by the Cohttp library. Developers just starting out want to explore their code interactively using a Unix-style development environment. The Cohttp library needs a TCP implementation to satisfy its module signature, which can be provided by the UnixSocket library.

When the programmers are satisfied that their HTTP logic is working, they can recompile to switch away from using Unix sockets to the OCaml TCP/IP stack shown by MirTCP in figure 5. This still requires a Unix kernel but only as a shell to deliver Ethernet frames to the Web-server process (which now incorporates an OCaml TCP/IP stack as part of the application). The last compilation strategy drops the dependency on Unix entirely and recompiles the MirNet module to link directly to a Xen network driver, which in turn pulls in all the dependencies it needs to boot on Xen. This progressive recompilation is key to the usability of MirageOS - since gradual evolution from the tried-and-tested Linux or FreeBSD functionality still ends up with specialized unikernels that can be deployed on the public cloud. This modular operating-system structure has led to a number of other back ends being implemented in a similar vein to Xen. MirageOS now has experimental back ends that implement a simulator in NS3 (for large-scale functional testing), a FreeBSD kernel module back end, and even a JavaScript target by using the js_of_ocaml compiler. A natural consequence of this modularity is that it is easier to write portable code that defines exactly what it needs from a target platform, which is increasingly difficult on modern operating systems with the lack of a modern equivalent of POSIX (which has led Linux,

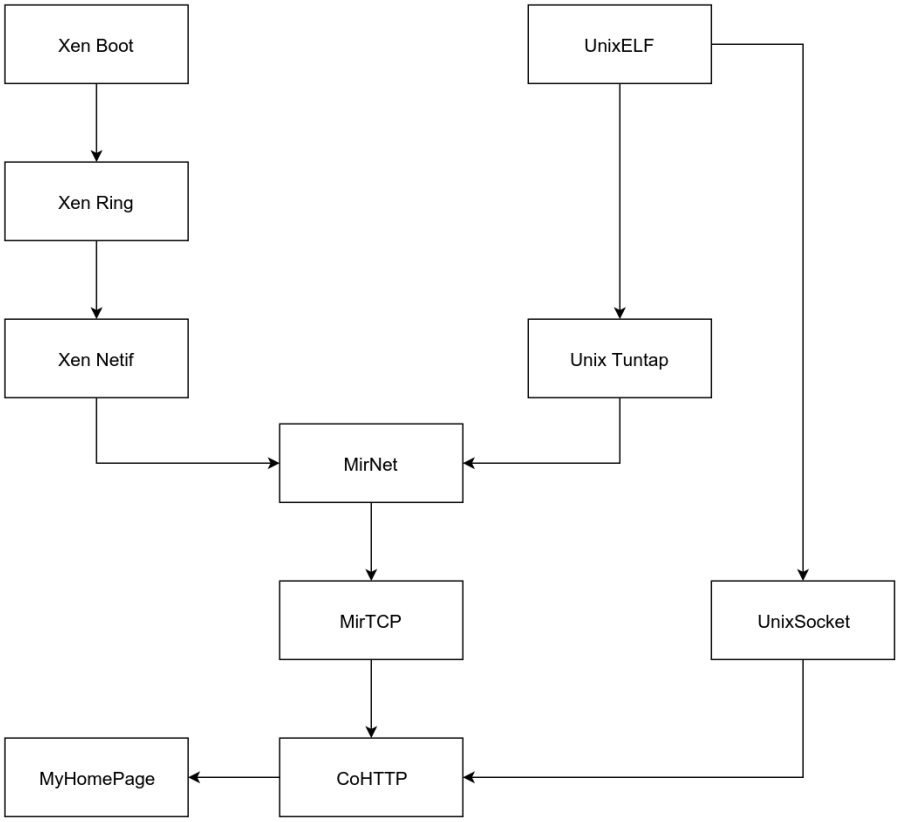

Figure 4: Partial module graph for a static web server

FreeBSD, Mac OS X, and Windows to have numerous incompatible APIs for highperformance services).

4.9 CONFIGURATION AND STATE

Libraries in MirageOS are designed in as functional a style as possible: they are reentrant with explicit state handles, which are in turn serializable so that they can be reconstructed explicitly. An application consists of a set of libraries plus some configuration code, all linked together. The configuration is structured as a tree roughly like a file system, with subdirectories being parsed by each library to initialize their own values (reminiscent of the Plan 9 operating system). All of this is connected by metaprogramming an OCaml program generates more OCaml code that is compiled until the desired target is reached.

The metaprogramming extends into storage as well. If an application uses a small set of files (which would normally require all the baggage of block devices and a file system), MirageOS can convert it into a static OCaml module that satisfies the file-system module signature, relieving it of the need for an external storage dependency. The entire MirageOS home page (http://openmirage.org) is served in this manner.

One (deliberate) consequence of metaprogramming is that large blocks of functionality may be entirely missing from the output binary. This makes dynamic reconfiguration of the most specialized targets impossible, and a configuration change requires the unikernel to be relinked. The lines of

active (i.e., post-configuration) code involved in a MirageOS Web server are shown in table 1, giving a sense of the small amount of code involved in such a recompilation.

Library	C / kLOC	OCaml / kLOC
boot	18	0
OCaml runtime	20	0
threads	5	27
interdomain comms	trace	1
network driver	0	1
TCP/IP	trace	12
block driver	0	1
HTTP	0	11
Total	43	52

Table 1: Approximate sizes of libraries used by a typical MirageOS Xen unikernel running a web server

4.10 LINKING THE XEN UNIKERNEL

In a conventional OS, application source code is first compiled into object files via a native-code compiler and then handed off to a linker that generates an executable binary. After compilation, a dynamic linker loads the executable and any shared libraries into a process with its own address space. The process can then communicate with the outside world by system calls, mediated by the operating-system kernel. Within the kernel, various subsystems such as the network stack or virtual memory system process system calls and interact with the hardware.

In MirageOS, the OCaml compiler receives the source code for an entire kernel's worth of code and links it into a stand-alone native-code object file. It is linked against a minimal runtime that provides boot support and the garbage collector. There is no preemptive threading, and the kernel is event-driven via an I/O loop that polls Xen devices.

The Xen unikernel compilation derives its performance benefit from the fact that the running kernel has a single virtual address space, designed to run only the OCaml runtime. The virtual address space of the MirageOS Xen unikernel target is shown in figure 5. Since all configuration information is explicitly part of the compilation, there is no longer a need for the usual dynamic linking support that requires executable mappings to be added after the VM has booted.

4.11 BENEFITS

Consider the life cycle of a traditional application. First the source code is compiled to a binary. Later, the binary is loaded into memory and an OS process is created to execute it. The first thing the running process will do is read its configuration file and specialize itself to the environment it finds itself in. Many different applications will run exactly the same binary, obtained from the same binary package, but with different configuration files. These configuration files are effectively additional program code, ex-

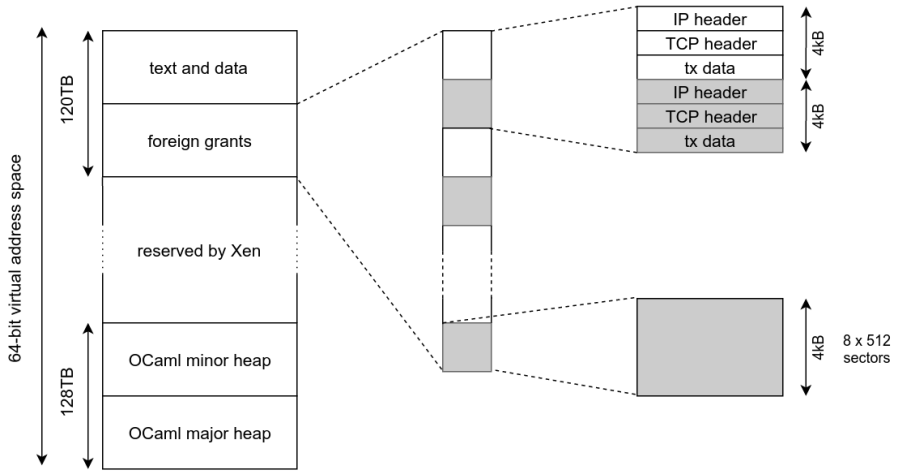

Figure 5: Virtual address space of the MirageOS Xen Unikernel Target

cept they're normally written in ad-hoc languages and interpreted at runtime rather than compiled.

4.12 DEPLOYMENT AND MANAGEMENT

Configuration is a considerable overhead in managing the deployment of a large cloud-hosted service. The traditional split between the compiled (code) and interpreted (configuration) is unnecessary with unikernel compilation. Application configuration is code—perhaps in an embedded domain-specific language—and the compiler can analyze and optimize across the whole unikernel.

In MirageOS, rather than treating the database, web server, etc. as independent applications that must be connected by configuration files, they are treated as libraries within a single application, allowing the application developer to configure them using either simple library calls for dynamic parameters or metaprogramming tools for static parameters. This has the useful effect of making configuration decisions explicit and programmable in a host language rather than manipulating many ad-hoc text files and thus benefiting from static-analysis tools and the compiler's type checker. The result is a big reduction in the effort needed to configure complex multiservice application VMs.

One downside to a unikernel is the burden it places on the cloud orchestration layers because of the need to schedule many more VMs with greater churn (since every reconfiguration requires the VM to be redeployed). The popular orchestration implementations have grown rather organically in recent years and consist of many distributed components that are not only difficult to manage, but also relatively high in latency and resource consumption.

One of the first production uses for MirageOS is to fix the cloud-management stacks by evolving the OCaml code within XenServer toward the structured unikernel worldview. This turns the monolithic management layer into a more agile set of intercommunicating VMs that can be scheduled and restarted independently. MirageOS makes constructing these single-purpose

VMs easy: they are first built and tested as regular Unix applications before flipping a switch and relinking against the Xen kernel libraries. When they are combined with Xen driver domains, they can dramatically increase the security and robustness of the cloud-management stack.

4.13 RESOURCE EFFICIENCY AND CUSTOMIZATION

The cloud is an environment where all resource usage is metered and rented. At the same time, multitenant services suffer from variability in load that encourages rapid scaling of deployments— both up to meet current demand and down to avoid wasting money. In MirageOS, features that are not used in a particular build are not included, and whole-system optimization techniques can be used to eliminate waste at compilation time rather than deployment time. In the most specialized mode, all configuration files are statically evaluated, enabling extensive dead-code elimination at the cost of having to recompile to reconfigure the service.

The small binary size of the unikernels (on the order of hundreds of kilobytes in many cases) makes deployment to remote data centers across the Internet much smoother. Boot time is also easily less than a second, making it feasible to boot a unikernel in response to incoming network packets.

The MLton compiler pioneered WPO (whole program optimization), where an application and all of its libraries are optimized together. In the libOS world, a whole program is actually a whole operating system: this technique can now optimize all the way from application-level code to low-level device drivers. Traditional systems eschew WPO in favor of dynamic linking, sometimes in combination with JIT (just-in-time) compiling, where a program is analyzed dynamically, and optimized code is generated on the fly. Whole-program, compile-time optimization is more appropriate for cloud applications that care about resource efficiency and reducing their attack surface. Other research elaborates on the security benefits.

An interesting recent trend is a move toward operating-system containers in which each container is managed by the same operating-system kernel but with an isolated file system, network, and process group. Containers are quick to create since there is no need to boot a new kernel, and they are fully compatible with existing kernel interfaces. However, these gains are made at the cost of reduced security and isolation; unikernels share only the minimal hypervisor services via a small API, which is easy to understand and audit. Unikernels demonstrate that layering language runtimes onto a hypervisor is a viable alternative to lightweight containers.

4.14 A NEW FRONTIER OF PORTABILITY

The structure of MirageOS libraries shown in figure 4 explicitly encodes what the library needs from its execution environment. While this has conventionally meant a Posix-like kernel and user space, it is now possible to compile OCaml into more foreign environments, including FreeBSD kernel modules, JavaScript running in the browser, or (as the Scala language does) directly targeting the JVM (Java Virtual Machine). Some care is still required for execution properties that are not abstractable in the OCaml type system.

For example, floating-point numbers are generally forbidden when running as a kernel module; thus, a modified compiler emits a type error if floating-point code is used when compiling for that hardware target.

Other third-party OCaml code often exhibits a similar structure, making it much easier to work under MirageOS. For example, Arakoon is a distributed key-value store that implements an efficient multi-Paxos consensus algorithm. The source-code patch to compile it under MirageOS touched just two files and was restricted to adding a new module definition that mapped the Arakoon back-end storage to the Xen block driver interface.

The libOS principles underlying MirageOS mean that it is not limited to running on a hypervisor platform—many of the libraries can be compiled to multiscale environments, ranging from ARM smartphones to bare-metal kernel modules.

5 ARCHITECTURE AND IMPLEMENTATION

5.1 ARCHITECTURE OVERVIEW

Dune is a VM manager for creating, launching, managing, and scaling up Mirage unikernels. It can be deployed in any environment which supports Xen hypervisor, which includes popular cloud services like Amazon EC2, IBM Softlayer, Rackspace Cloud, Linode, etc.

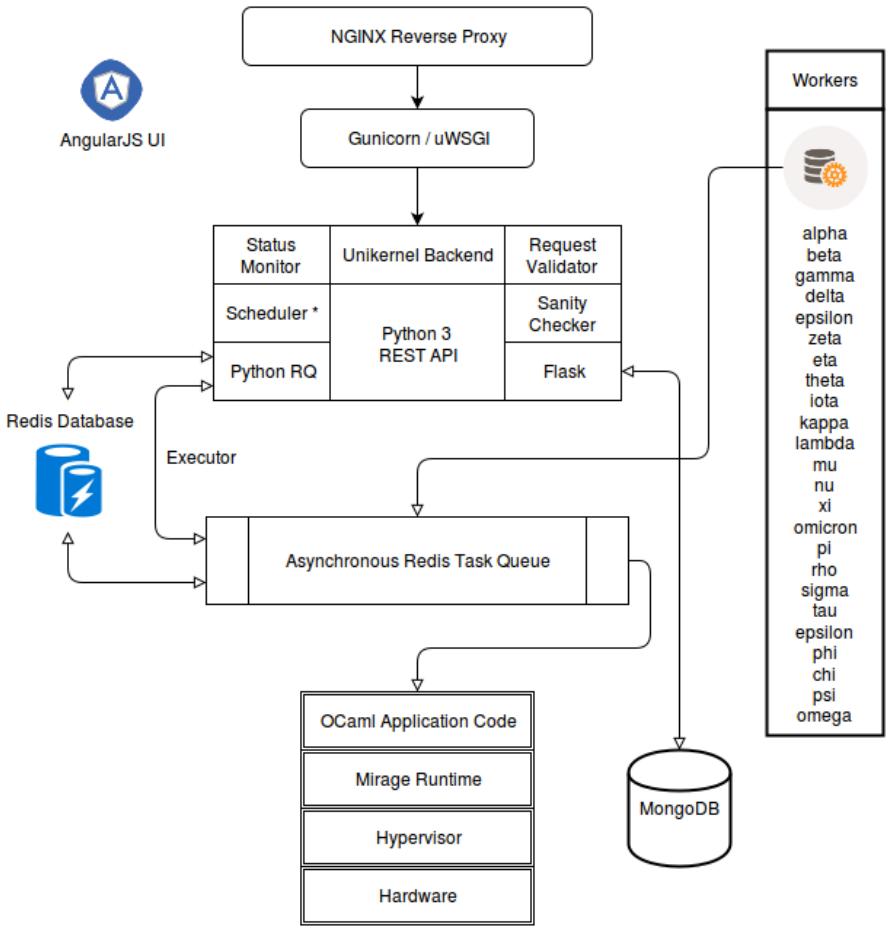

Figure 6: Architectural overview of Dune

5.2 DUNE API

The entry point to Dune is the API server powered by Python Flask. At startup, the REST API invokes the Sanity Checker to ensure all components of Dune are running, and properly configured. An list of sanity checks performed by Dune, in order of execution, are given below:

1. **check_environment** — Checks if "opam config env" has set all the required environment variables in the current shell.

   Raises: OPAMConfigurationError

2. **check_mirage** — Check if MirageOS is installed by "opam"

   Raises: UnikernelLibraryNotFound

3. **is_not_root** — Checks if current user is NOT "root"

   Raises: ExcessivePrivilegeError

4. **check_privilege** — Check if current user can execute apt-get on host without root access.

   Raises: InsufficientPrivilegeError

5. **check_privilege** — Check if current user can execute apt-get on host without root access.

   Raises: InsufficientPrivilegeError

6. **check_redis_server** — Check if instance of Redis server is running.

   Raises: RedisServerNotFound

7. **check_mongod_server** — Check running instance of MongoDB server.

   Raises: MongoDBServerNotFound

8. **check_redis_queue** — Check Python Redis Queue worker

   Raises: RedisQueueException

5.3 FLASK API ENDPOINTS

The API server instantiates a singleton object, which initializes the Flask application, and a MongoDB client.

1. `GET /api/unikernel/create` — Validate POST request data and enqueue jobs in the Redis Queue.
2. `GET /api/unikernel/<_id>` — Probe running status of a previously registered unikernel.
3. `GET /static/<_id>/<file>` — Serve text assets on the Flask webserver.

Example HTTP POST request to create a MirageOS unikernel

```
POST /api/unikernels/create HTTP/1.1
Host: 127.0.0.1:5000
Content-Type: application/json

{
    "meta": {
        "name": "Description of the unikernel",
        "project": "Name of your project",
        "module": ["module", "sub-module"]
    },
    "unikernel": "<unikernel.ml>",
    "config": "<config.ml>",
```

```
    "backend": "xen",
    "cron": "00 09-18 * * *"
}

HTTP/1.1 201 Created
Content-Type: application/json

{
    "id": 78217
}
```

5.4 BACKENDS

Dune supports two backends for running MirageOS unikernels — unix, and xen. Backends for Dune implement the `IUnikernelBackend` interface with member methods to register, configure, compile, optimize, start, stop, and destroy unikernels. Each of these method calls (except register) are executed within the context of a message broker queue. Dune uses Python Redis Queue as the message broker for asynchronous task execution. Every call through the unikernel backend must create an instance of the Executor class specifying the working directory for execution. Executor instances can perform logged calls using the Python subprocess library. It also provides interface to probe status of unikernel execution. Valid statuses recognized by Dune are `WAITING`, `RUNNING`, `SUCCEEDED`, `FAILED`, and `KILLED`. The backend uses MongoDB for persistence of unikernel status. This status is also reflected in the `/api/unikernel/<_id>` endpoint.

5.5 WORKERS

A worker within the context of Dune is an executor unit that is responsible for executing all the tasks in the job queue asynchronously. The Worker class is responsible for assigning workers to the message broker, names of which are mentioned in the architecture diagram. Another function which a Worker does is launching Redis Queue workers as daemons, tracking their PID, and storing debug level temporary logs. The Worker uses Python library Daemonize in order to achieve this. A very important thing to note is that function calls lose context, and state when executed inside the task queue. For this reason, all state information need to be passed to the task queue during the function call. All instances of database, and API need to be instantiated again inside the task queue.

```python
def launch_rq_worker() -> None:
    """
    Blocking function to launch a worker using Python RQ's
        internal API
    """
    with Connection():
        w = Worker(
            get_available_rq_worker_name()
        )
```

```
        w.work()


def launch_rq_daemon():
    worker = get_available_rq_worker_name()
    pid = "/tmp/%s.pid" % worker

    ...

    keep_fds = [fh.stream.fileno()]

    daemon = Daemonize(
        app="dune",
        pid=pid,
        action=launch_rq_worker,
        keep_fds=keep_fds
    )

    daemon.start()
```

5.6 TEST UNIKERNEL

For our implementation as well as benchmarks presented in the next chapter, we have used a console MirageOS unikernel that prints "hello world" six times, sleeping for 1 second in between. The unikernel program and its corresponding configuration file in OCaml are given below:

unikernel.ml

```
open Lwt

module Main (C: V1_LWT.CONSOLE) = struct

  let start c =
    for_lwt i = 0 to 6 do
      C.log c "hello world" ;
      lwt () = OS.Time.sleep 1.0 in
      return ()
    done

end
```

config.ml

```
open Mirage

let main =
  foreign "Unikernel.Main" (console @-> job)

let () =
  register "console" [main $ default_console]
```

6 RESULTS AND ANALYSIS

6.1 PERFORMANCE ANALYSIS

In this chapter, the exploratory benchmark experiments are presented which give an insight about the performance of a MirageOS unikernel program. Table 2 contains the precise execution times of various unikernel operations in milliseconds. It can be seen that unikernels boot up very quickly, and the boot time of 60ms is negligible compared to a relatively expensive operation. Furthermore, it must be noted that sum of the configure, compile times are a one-time expense, and are not incurred after the unikernel is generated.

Operation	Execution Time
configure	7836.333
compile	2623.191
boot	61.840
task	6000.00
start	6061.840
bootstrapping overhead	10521.364
spawning overhead	61.840

Table 2: Execution time (in ms) of Unikernel backend methods in MirageOS

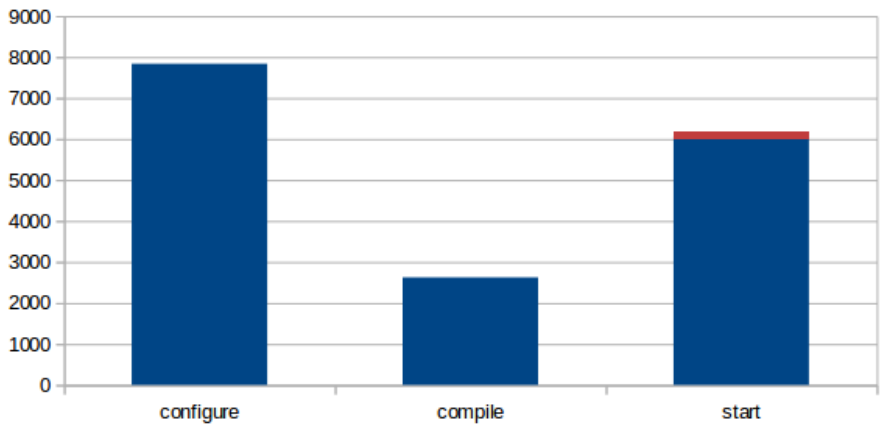

Figure 7: Execution time (in ms) of Unikernel backend methods in MirageOS

Figure 7 gives a comparison of execution times of various UNIX backend operations in a unikernel that prints "hello world" six times, sleeping 1 second in between.

6.2 CASE STUDY

Consider a data center interested in running a backup daemon which lasts for a total of 6 hours throughout the 24 hour time period of a day. For com-

parative purpose, we consider the case of a Digital Ocean droplet against unikernel. Hourly cost of a Digital Ocean droplet with 2GB memory, 2 core processor, 40GB SSD disk, and 3TB tansfer is 0.03 USD/hour.

Unikernels can be instructed to sleep during idle hours, and wake up on demand only to perform a particular job. The sleep and wake up pattern can be triggered using UNIX Cron expressions supported by Dune. The total compute hours of a unikernel, and a digital ocean droplet are shown in Figure 8. Associated prices per month are shown in Figure **??**. It is assumed that hourly rate of unikernel cloud hosting, and Digital Ocean droplet are the same, i.e., 0.03 USD/hour.

Daemon	Backup hrs	Idle hrs	Overhead	Wake hrs (alive)
Unikernel	180	0	1	181
DO droplet	180	540	0	720

Table 3: Monthly compute hours

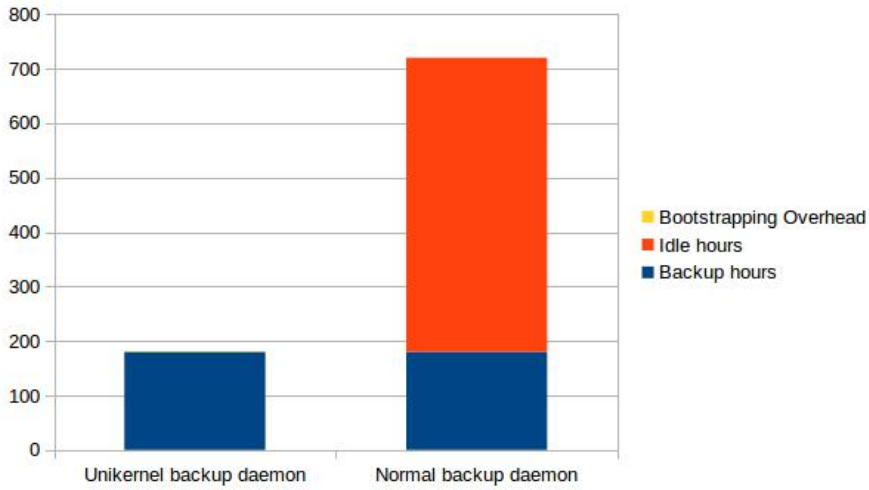

Figure 8: Monthly compute usage hours

Daemon	Wake hours (alive)	Monthly costs (in $)
Unikernel	181	5.43
DO droplet	720	21.6

Table 4: Monthly expenses per usage

7 CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

The cloud industry's reliance increases on containers and service-based applications, fueled by demands of performance, security, scaling and uptime requirements.

With the use of unikernels, RESTful interfaces, NoSQL and in-memory databases this thesis illustrated that it is possible to build solutions that provision specialized virtual machines. They have a single purpose – to run a single application that can be controlled remotely. These machines showed high performance, yet are lean in terms of memory footprint, with intrinsic security properties.

We also showed how unikernels can be used as application containers that can be executed remotely in a cloud platform supporting Xen. Using unikernels along with a VM manager such as Dune, instead of a full-blown traditional operating system, can reduce bills significantly in PaaS solutions with hourly rates.

7.2 FUTURE WORK

Dune can be extended to support other unikernel flavours like HaLVM, Clive, OSv, Drawbridge, IncludeOS, runtime.js, etc. Adding a new unikernel backend involves implementing the `IUnikernelBackend` interface.

Some options to be explored remain, such as implementation of an appropriate scheduling algorithm and improvements to horizontal scalability of Dune. Implementation of modules for adding self-healing, binpacking, and load balancing capabilities will make Dune suitable for deployment in production environment. With a view to improving this software suite, the source code has been released under the open source Apache License Version 2.

BIBLIOGRAPHY

Anil Madhavapeddy, et al.

2015 *Towards Functional Operating Systems*, http://decks.openmirage.org/bob15#/.

Bratterud, Alfred, Alf-Andre Walla, Paal E Engelstad, Kyrre Begnum, et al.

2015 "IncludeOS: A minimal, resource efficient unikernel for cloud services," in *2015 IEEE 7th International Conference on Cloud Computing Technology and Science (CloudCom)*, IEEE, pp. 250-257.

Kantee, Antti et al.

2012 "Flexible operating system internals: the design and implementation of the anykernel and rump kernels."

Kivity, Avi, Dor Laor, Glauber Costa, Pekka Enberg, Nadav Har'El, Don Marti, and Vlad Zolotarov

2014 "OSv—optimizing the operating system for virtual machines," in *2014 usenix annual technical conference (usenix atc 14)*, pp. 61-72.

Madhavapeddy, Anil, Richard Mortier, Charalampos Rotsos, David Scott, Balraj Singh, Thomas Gazagnaire, Steven Smith, Steven Hand, and Jon Crowcroft

2013 "Unikernels: Library operating systems for the cloud," in *ACM SIGPLAN Notices*, 4, ACM, vol. 48, pp. 461-472.

Madhavapeddy, Anil and David J Scott

2013 "Unikernels: Rise of the virtual library operating system," *Queue*, 11, 11, p. 30.